UNIVERSIDAD TECNOLÓGICA DE EL SALVADOR

Técnicas de Producción Industrial de Software I

Ciclo 02-2026 · Tercer período

DOCUMENTO DE EVIDENCIAS · SEMANA 12

Práctica integradora: tablero de diagnóstico del dispositivo

Lectura, interpretación y control del ciclo de vida de sensores en RegistroIncidencias

Estudiante	Henry Buendia
Tipo de actividad	Práctica integradora de preparación
Proyecto	RegistroIncidencias (Android · Kotlin · Jetpack Compose)
Repositorio	https://github.com/HABA03/RegistroIncidencias
Rama	master
Entorno de prueba	Emulador Pixel 10a · Android 17 (API 37)
Fecha	18 de octubre de 2026

1. Objetivo y alcance

Integrar en RegistroIncidencias una sección de diagnóstico que muestre el estado de los sensores del equipo, siguiendo el flujo completo solicitado: comprobar disponibilidad, escuchar cambios, actualizar el estado de la interfaz, mostrar una interpretación comprensible y liberar el recurso cuando deja de utilizarse. Antes de modificar el proyecto se verificó que la versión anterior compilaba correctamente (./gradlew assembleDebug).

La funcionalidad se presenta como una pestaña «Diagnóstico» junto a la pestaña «Registro». Así, antes de reportar una falla de hardware, la persona usuaria puede confirmar si el sensor responde. El formulario de registro conserva lo escrito al cambiar de pestaña.

2. Recursos seleccionados

Sensor	Constante de Android	Qué permite diagnosticar
Acelerómetro	TYPE_ACCELEROMETER	Si el equipo está quieto, en movimiento leve o recibe una sacudida.
Sensor de luz	TYPE_LIGHT	Nivel de iluminación del entorno, usado para el brillo automático.
Sensor de proximidad	TYPE_PROXIMITY	Si hay un objeto cerca de la pantalla (por ejemplo, la oreja).

Tabla 1. Sensores revisados por la pantalla de diagnóstico.

3. Implementación técnica

La implementación sigue las cinco etapas del flujo propuesto en la guía:

Etapa	Implementación en el proyecto
Disponibilidad	SensorManager.getDefaultSensor(tipo). Si devuelve null, la tarjeta muestra «No disponible» y un mensaje claro; si no existe ningún sensor, el botón de lectura se desactiva.
Escucha	SensorEventListener con SENSOR_DELAY_UI (≈ 15 lecturas por segundo): suficiente para la interfaz y de menor consumo.
Estado	LecturaSensorState guarda la última lectura, si el listener está activo y los eventos recibidos.
Interpretación	InterpretacionSensor convierte los valores en frases con contexto; los números solo acompañan.

Etapa	Implementación en el proyecto
Ciclo de vida	LifecycleResumeEffect registra el listener en ON_RESUME y lo libera en ON_PAUSE o al salir de la pestaña. El botón «Pausar lectura» también libera los sensores.

Tabla 2. Flujo implementado para cada sensor.

Archivo	Cambio
LecturaSensor.kt sensores/	Nuevo. Registro y liberación del listener según el ciclo de vida.
InterpretacionSensor.kt sensores/	Nuevo. Reglas de interpretación sin dependencias de Android.
DiagnosticoDispositivoScreen.kt ui/diagnostico/	Nuevo. Panel de monitoreo y tarjetas por sensor.
RecursoSensor.kt model/	Nuevo. Catálogo de sensores revisados.
MainActivity.kt	Pestañas Registro/Diagnóstico y estado guardable del formulario.
strings.xml res/values/	Textos de la pantalla, interpretaciones y plurales.

Tabla 3. Archivos creados o modificados (carpetas relativas a app/src/main).

4. Interpretación de las lecturas

Cada lectura se clasifica con reglas documentadas. En el acelerómetro se mide cuánto se aleja la magnitud del vector de la gravedad (9.81 m/s²), que es lo único que registra en reposo.

Sensor	Regla	Mensaje mostrado
Acelerómetro	Desviación < 0.6 m/s²	Dispositivo en reposo
	Desviación < 3.0 m/s²	Movimiento leve detectado
	Desviación ≥ 3.0 m/s²	Movimiento brusco detectado (sacudida o golpe)
Luz	< 10 lx · < 200 lx · < 1 000 lx	Ambiente oscuro · Iluminación tenue · Iluminación interior adecuada
	< 10 000 lx · ≥ 10 000 lx	Ambiente con mucha luz · Luz solar directa
Proximidad	Distancia < mín(rango máximo, 5 cm)	Objeto cerca de la pantalla · Sin objetos cerca

Tabla 4. Reglas de interpretación (validadas con 10 pruebas unitarias).

5. Evidencia en ejecución

Las capturas se obtuvieron en el emulador. Los valores de los sensores se simularon con la consola del emulador (adb emu sensor set), que equivale al panel Extended controls › Virtual sensors de Android Studio.

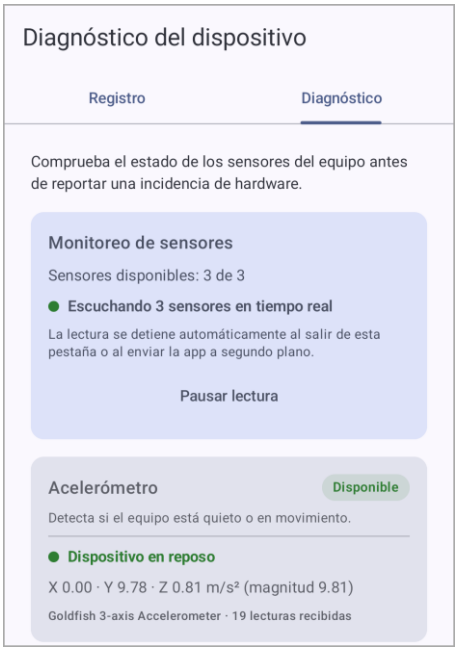


Figura 1. Tres sensores disponibles y acelerómetro en reposo.



Figura 2. Valores modificados: movimiento brusco, luz solar directa y objeto cerca.

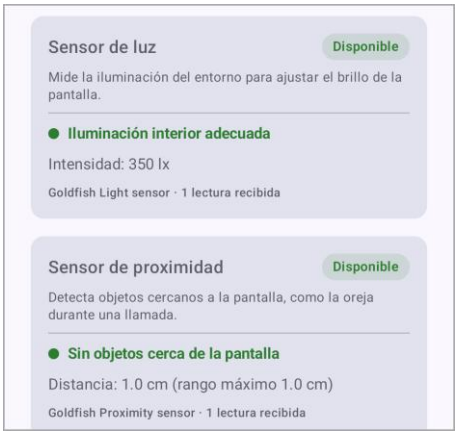


Figura 3. Iluminación interior y proximidad sin objetos cerca.



Figura 4. El formulario conserva lo escrito al volver de la pestaña Diagnóstico.



Figura 5. Lectura en pausa: sensores liberados y última lectura conservada.

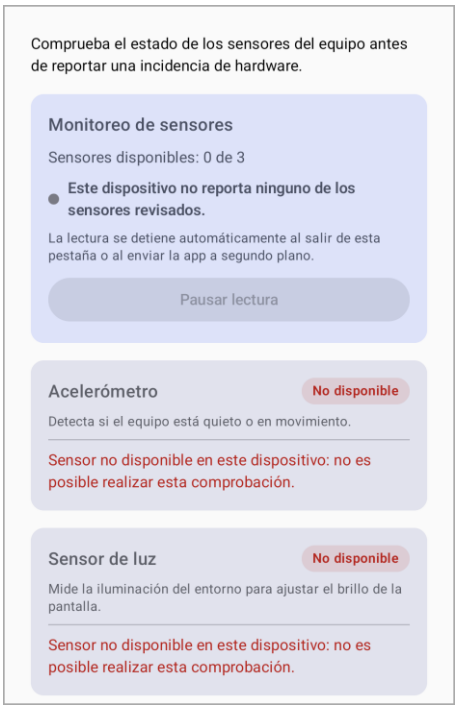


Figura 6. Mensaje claro cuando el equipo no tiene sensores disponibles.

6. Control del ciclo de vida y liberación del recurso

Ningún sensor queda escuchando de forma indefinida. Cada registro y cada liberación dejan una línea en Logcat con la etiqueta DiagnosticoSensores. La siguiente secuencia se obtuvo durante la prueba instrumentada CicloVidaSensoresTest:

Hora	Evento	Resultado en Logcat
21:55:06	Se abre la pestaña Diagnóstico	Listener registrado (3 sensores)
21:55:09	La app deja el primer plano (ON_PAUSE)	Listener liberado (3 sensores)
21:55:11	La app vuelve al primer plano (ON_RESUME)	Listener registrado (3 sensores)
21:55:14	Cambio a la pestaña Registro	Listener liberado (3 sensores)
21:55:15	Regreso a la pestaña Diagnóstico	Listener registrado (3 sensores)
21:55:16	Fin de la prueba (ON_PAUSE)	Listener liberado (3 sensores)

Tabla 5. Registro y liberación observados (archivo evidencia/20_logcat_ciclo_sensores.txt).

La Figura 5 muestra el mismo comportamiento desde la interfaz: al pausar, el contador de lecturas recibidas deja de aumentar y se conserva la última lectura con una nota aclaratoria.

7. Verificación automatizada

Prueba	Qué verifica	Aprobadas
InterpretacionSensorTest Unitaria	Umbrales de movimiento, luz y proximidad, incluidos los valores límite.	10 de 10
DiagnosticoDispositivoScreenTest Instrumentada	Mensaje sin sensores, pausa y reanudación, formulario conservado.	3 de 3
CicloVidaSensoresTest Instrumentada	Liberación en ON_PAUSE y nuevo registro en ON_RESUME.	1 de 1
RegistroIncidenciasScreenTest Instrumentada (más prueba base)	Regresión de las funciones de semanas anteriores.	6 de 6

Tabla 6. Resultado de ./gradlew testDebugUnitTest connectedDebugAndroidTest.

8. Limitaciones

- **Sensores virtuales:** los valores se introducen manualmente y no tienen el ruido de un sensor real, por lo que los umbrales deben calibrarse en un teléfono.
- **Proximidad:** el sensor del emulador es binario (0 o 1 cm de rango máximo), como en muchos teléfonos, y no mide distancias intermedias.
- **Acelerómetro:** la lectura incluye la gravedad; un giro lento del equipo no se interpreta como movimiento, solo los cambios bruscos de aceleración.
- **Capturas:** con aceleración gráfica del equipo anfitrión, adb shell screencap devolvía imágenes negras; las capturas se tomaron con captureToImage() de Compose Test.

9. Pruebas propuestas en un dispositivo físico

- **Movimiento:** dejar el teléfono sobre una mesa, caminar con él y agitarlo, para ajustar los umbrales de reposo y movimiento.
- **Iluminación:** medir en una habitación oscura, en una oficina y en exteriores con luz solar.
- **Proximidad:** cubrir el sensor con la mano y acercar el teléfono a la oreja.
- **Consumo:** confirmar con adb shell dumpsys sensorservice y con las estadísticas de batería que no queda ningún listener activo al bloquear la pantalla.

10. Correspondencia con la rúbrica

Criterio	Puntos	Dónde se evidencia
Disponibilidad y lectura del recurso	25	Secciones 2 y 3; Figuras 1, 2, 3 y 6.
Interpretación visible para el usuario	25	Sección 4; Figuras 1, 2, 3 y 5.
Control de ciclo / liberación del recurso	20	Sección 6; Figura 5; Tabla 5.
Evidencia técnica y repositorio	20	Secciones 5 y 7; README y carpeta evidencia/ en GitHub.
Explicación clara y honesta de limitaciones	10	Secciones 8 y 9.

Tabla 7. Relación entre la rúbrica sugerida y este documento.

11. Respuesta personal

¿Qué pregunta del parcial te ayuda a preparar esta práctica? Me ayuda a preparar la pregunta sobre cómo integrar y gestionar un recurso del dispositivo en una aplicación móvil: comprobar que existe, escuchar sus cambios, reflejarlos en el estado de la interfaz y liberarlo según el ciclo de vida para no consumir batería de forma innecesaria.